

```

# =====
# GEOSPATIAL INCIDENT RESPONSE SYSTEM
# Trend Analytics: Champion Model Selection
# Algorithms: Moving Average vs SARIMA vs Prophet
# Scope: Negros Island Region (NIR), Philippines
# =====

# -----
# CELL 1: INSTALL DEPENDENCIES
# Run this first, then restart the kernel
# -----
# !pip install prophet statsmodels scikit-learn folium matplotlib seaborn
pandas numpy openpyxl

# -----
# CELL 2: IMPORT LIBRARIES
# -----
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from statsmodels.tsa.statespace.sarimax import SARIMAX
from statsmodels.tsa.seasonal import seasonal_decompose
from prophet import Prophet

from sklearn.metrics import mean_absolute_error, mean_squared_error

print("✅ All libraries imported successfully!")

# -----

```

```

# CELL 3: GENERATE SAMPLE DATASET
# (Replace this with your actual NIR dataset)
# If you have a CSV, use:
# df_raw = pd.read_csv('your_file.csv')
# _____

np.random.seed(42)

# Generate 3 years of daily incident data (2021-2023)
date_range = pd.date_range(start='2021-01-01', end='2023-12-31', freq='D')
n = len(date_range)

# Simulate realistic incident patterns with seasonality + noise
trend = np.linspace(5, 15, n) # slow upward trend
seasonality = 4 * np.sin(2 * np.pi * np.arange(n) / 365) # yearly seasonality
weekly = 2 * np.sin(2 * np.pi * np.arange(n) / 7) # weekly pattern
noise = np.random.normal(0, 2, n) # random noise

incident_counts = np.clip(trend + seasonality + weekly + noise, 1,
None).astype(int)

# Assign incident types randomly
incident_types = np.random.choice(
    ['Fire', 'Crime', 'Accident'],
    size=n,
    p=[0.3, 0.45, 0.25]
)

# Simulate NIR latitude/longitude (Bacolod City area)
latitudes = np.random.uniform(10.55, 10.75, n)
longitudes = np.random.uniform(122.85, 123.05, n)

df_raw = pd.DataFrame({
    'date': date_range,
    'incident_type': incident_types,

```

```

        'count':      incident_counts,
        'latitude':    latitudes,
        'longitude':   longitudes,
        'barangay':
np.random.choice(['Mandalagan', 'Handumanan', 'Bata', 'Singcang', 'Villamonte', 'Bana
nago'], size=n),
        'cause':
np.random.choice(['Electrical', 'Negligence', 'Unknown', 'Intentional'], size=n),
    })

```

```

print(f"✅ Dataset created: {len(df_raw)} records from
{df_raw['date'].min().date()} to {df_raw['date'].max().date()}")
print(f"\n📋 Sample Data:")
print(df_raw.head(10).to_string(index=False))

```

```

# -----
# CELL 4: DATA PREPARATION & CLEANING
# -----
print("=" * 55)
print("  STEP 1: DATA PREPARATION & CLEANING")
print("=" * 55)

```

```
df = df_raw.copy()
```

```

# Step 1a: Convert date to datetime
df['date'] = pd.to_datetime(df['date'])

```

```

# Step 1b: Check for missing values
print(f"\n🔍 Missing Values Check:")
print(df.isnull().sum())

```

```

# Step 1c: Drop duplicates
before = len(df)
df.drop_duplicates(inplace=True)
print(f"\n✂ Duplicates removed: {before - len(df)}")

```

```

# Step 1d: Add useful time features
df['year']      = df['date'].dt.year
df['month']     = df['date'].dt.month
df['day_of_week'] = df['date'].dt.day_name()
df['week']      = df['date'].dt.isocalendar().week

# Step 1e: Aggregate daily total incidents
df_daily = df.groupby('date')['count'].sum().reset_index()
df_daily.columns = ['date', 'total_incidents']
df_daily = df_daily.sort_values('date').reset_index(drop=True)

print(f"\n✅ Cleaned daily aggregated data: {len(df_daily)} rows")
print(f"\n📋 Daily Incident Summary:")
print(df_daily.describe().round(2))

# -----
# CELL 5: EXPLORATORY DATA ANALYSIS (EDA)
# -----

print("=" * 55)
print("  STEP 2: EXPLORATORY DATA ANALYSIS")
print("=" * 55)

fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('NIR Incident Data – Exploratory Analysis', fontsize=16,
fontweight='bold', y=1.01)

# Plot 1: Daily incident trend over time
axes[0, 0].plot(df_daily['date'], df_daily['total_incidents'],
color='steelblue', linewidth=0.8, alpha=0.7)
axes[0, 0].set_title('Daily Incident Count Over Time', fontweight='bold')
axes[0, 0].set_xlabel('Date')
axes[0, 0].set_ylabel('Number of Incidents')
axes[0, 0].xaxis.set_major_formatter(mdates.DateFormatter('%Y-%m'))
axes[0, 0].xaxis.set_major_locator(mdates.MonthLocator(interval=3))

```

```

plt.setp(axes[0, 0].xaxis.get_majorticklabels(), rotation=45)

# Plot 2: Incidents by type
type_counts = df.groupby('incident_type')['count'].sum()
colors = ['#e74c3c', '#3498db', '#f39c12']
axes[0, 1].pie(type_counts, labels=type_counts.index, autopct='%1.1f%%',
               colors=colors, startangle=90)
axes[0, 1].set_title('Incidents by Type', fontweight='bold')

# Plot 3: Monthly average
monthly_avg = df.groupby(df['date'].dt.month)['count'].mean()
month_names =
['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec']
axes[1, 0].bar(month_names, monthly_avg.values, color='steelblue',
               edgecolor='white')
axes[1, 0].set_title('Average Incidents by Month', fontweight='bold')
axes[1, 0].set_xlabel('Month')
axes[1, 0].set_ylabel('Average Incidents')

# Plot 4: Incidents by day of week
day_order =
['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday']
daily_avg = df.groupby('day_of_week')['count'].mean().reindex(day_order)
axes[1, 1].bar(day_order, daily_avg.values, color='coral', edgecolor='white')
axes[1, 1].set_title('Average Incidents by Day of Week', fontweight='bold')
axes[1, 1].set_xlabel('Day')
axes[1, 1].set_ylabel('Average Incidents')
plt.setp(axes[1, 1].xaxis.get_majorticklabels(), rotation=30)

plt.tight_layout()
plt.savefig('eda_analysis.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ EDA charts displayed!")

```

```

# _____

```

```

# CELL 6: SEASONAL DECOMPOSITION
# -----
print("=" * 55)
print("  STEP 3: TREND DECOMPOSITION")
print("=" * 55)

# Set date as index for decomposition
ts = df_daily.set_index('date')['total_incidents']

decomposition = seasonal_decompose(ts, model='additive', period=30)

fig, (ax1, ax2, ax3, ax4) = plt.subplots(4, 1, figsize=(14, 10))
fig.suptitle('Incident Time Series Decomposition', fontsize=14,
fontweight='bold')

decomposition.observed.plot(ax=ax1, color='steelblue')
ax1.set_title('Observed')
ax1.set_ylabel('Count')

decomposition.trend.plot(ax=ax2, color='green')
ax2.set_title('Trend')
ax2.set_ylabel('Count')

decomposition.seasonal.plot(ax=ax3, color='orange')
ax3.set_title('Seasonality')
ax3.set_ylabel('Count')

decomposition.resid.plot(ax=ax4, color='red', alpha=0.6)
ax4.set_title('Residual (Noise)')
ax4.set_ylabel('Count')

plt.tight_layout()
plt.savefig('decomposition.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Decomposition complete – you can now see Trend, Seasonality, and
Noise separately!")

```

```

# -----
# CELL 7: TRAIN / TEST SPLIT
# -----

print("=" * 55)
print(" STEP 4: TRAIN / TEST SPLIT (80/20)")
print("=" * 55)

split_index = int(len(df_daily) * 0.80)
train_df     = df_daily[:split_index].copy()
test_df      = df_daily[split_index:].copy()

print(f"✅ Training set : {len(train_df)} days
({train_df['date'].min().date()} → {train_df['date'].max().date()})")
print(f"✅ Testing set  : {len(test_df)} days
({test_df['date'].min().date()} → {test_df['date'].max().date()})")

# Visualize the split
plt.figure(figsize=(14, 5))
plt.plot(train_df['date'], train_df['total_incidents'], label='Training Data
(80%)', color='steelblue')
plt.plot(test_df['date'], test_df['total_incidents'], label='Testing Data
(20%)', color='orange')
plt.axvline(x=train_df['date'].iloc[-1], color='red', linestyle='--',
linewidth=1.5, label='Split Point')
plt.title('Train / Test Split Visualization', fontsize=14, fontweight='bold')
plt.xlabel('Date')
plt.ylabel('Incident Count')
plt.legend()
plt.tight_layout()
plt.savefig('train_test_split.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Train/Test split done!")

```

```

# -----
# CELL 8: ALGORITHM 1 – MOVING AVERAGE (Baseline)
# -----

print("=" * 55)
print("  STEP 5A: ALGORITHM 1 – MOVING AVERAGE")
print("=" * 55)

window = 7  # 7-day moving average

# Apply moving average on training data
train_ma = train_df.copy()
train_ma['ma_prediction'] =
train_ma['total_incidents'].rolling(window=window).mean()

# For test predictions, use the last moving average value from training
last_ma_value = train_ma['ma_prediction'].dropna().iloc[-1]
ma_predictions = np.full(len(test_df), last_ma_value)

print(f"✅ Moving Average (window={window} days) applied!")
print(f"  Last training MA value: {last_ma_value:.2f}")
print(f"  Predicting {len(test_df)} test days with constant MA forecast")

# -----
# CELL 9: ALGORITHM 2 – SARIMA
# -----

print("=" * 55)
print("  STEP 5B: ALGORITHM 2 – SARIMA")
print("=" * 55)

train_series = train_df.set_index('date')['total_incidents']

print("🕒 Fitting SARIMA model... (this may take 30–60 seconds)")

sarima_model = SARIMAX(
    train_series,

```



```

    order=(1, 1, 1),          # p=1 (AR), d=1 (diff), q=1 (MA)
    seasonal_order=(1, 1, 0, 7), # weekly seasonality
    enforce_stationarity=False,
    enforce_invertibility=False
)
sarima_result = sarima_model.fit(dispatch=False)
sarima_predictions = sarima_result.forecast(steps=len(test_df))
sarima_predictions = np.clip(sarima_predictions.values, 0, None)

print("✅ SARIMA model fitted and predictions generated!")
print(f"    AIC Score: {sarima_result.aic:.2f}")

# -----
# CELL 10: ALGORITHM 3 – PROPHET
# -----

print("=" * 55)
print("    STEP 5C: ALGORITHM 3 – PROPHET")
print("=" * 55)

# Prophet requires 'ds' and 'y' columns
train_prophet = train_df.rename(columns={'date': 'ds', 'total_incidents':
'y'})
test_prophet = test_df.rename(columns={'date': 'ds', 'total_incidents': 'y'})

print("🕒 Fitting Prophet model...")

prophet_model = Prophet(
    yearly_seasonality=True,
    weekly_seasonality=True,
    daily_seasonality=False,
    changepoint_prior_scale=0.05, # controls flexibility of trend
    seasonality_prior_scale=10,
)

# Add Philippine holidays context (manual)

```

```

prophet_model.add_country_holidays(country_name='PH')

prophet_model.fit(train_prophet)

# Predict for test period
future          = prophet_model.make_future_dataframe(periods=len(test_df))
forecast         = prophet_model.predict(future)
prophet_pred_df = forecast[forecast['ds'].isin(test_df['date'])]
prophet_predictions = np.clip(prophet_pred_df['yhat'].values, 0, None)

print("✅ Prophet model fitted and predictions generated!")
print(f"    Forecast period: {len(prophet_predictions)} days")

# -----
# CELL 11: EVALUATE ALL 3 MODELS
# -----
print("=" * 55)
print("    STEP 6: MODEL EVALUATION")
print("=" * 55)

actual = test_df['total_incidents'].values

def evaluate_model(actual, predicted, model_name):
    mae = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))
    # Avoid division by zero in MAPE
    mask = actual != 0
    mape = np.mean(np.abs((actual[mask] - predicted[mask]) / actual[mask])) *
100
    return {'Model': model_name, 'MAE': round(mae, 4), 'RMSE': round(rmse, 4),
'MAPE (%)': round(mape, 4)}

results = [
    evaluate_model(actual, ma_predictions, "Moving Average"),
    evaluate_model(actual, sarima_predictions, "SARIMA"),

```

```

    evaluate_model(actual, prophet_predictions, "Prophet"),
]

results_df = pd.DataFrame(results)
results_df = results_df.sort_values('RMSE') # sort by best RMSE

print("\n📊 Model Evaluation Results:")
print(results_df.to_string(index=False))

# Determine champion
champion = results_df.iloc[0]['Model']
print(f"\n🏆 CHAMPION MODEL: {champion}")
print(f"    (Lowest RMSE = {results_df.iloc[0]['RMSE']})")

# -----
# CELL 12: EVALUATION METRICS BAR CHART
# -----

print("=" * 55)
print("  STEP 7: EVALUATION METRICS VISUALIZATION")
print("=" * 55)

metrics    = ['MAE', 'RMSE', 'MAPE (%)']
models     = results_df['Model'].tolist()
colors     = ['#e74c3c', '#f39c12', '#2ecc71']

fig, axes = plt.subplots(1, 3, figsize=(15, 5))
fig.suptitle('Model Comparison - Evaluation Metrics\n(Lower is Better ✅)',
             fontsize=14, fontweight='bold')

for i, metric in enumerate(metrics):
    values = results_df[metric].tolist()
    bars   = axes[i].bar(models, values, color=colors, edgecolor='white',
width=0.5)
    axes[i].set_title(metric, fontweight='bold', fontsize=13)
    axes[i].set_ylabel('Score')

```

```

axes[i].set_ylim(0, max(values) * 1.3)

# Label bars
for bar, val in zip(bars, values):
    axes[i].text(bar.get_x() + bar.get_width()/2, bar.get_height() +
max(values)*0.02,
                  f'{val:.2f}', ha='center', va='bottom',
fontweight='bold', fontsize=11)

# Highlight winner
min_idx = values.index(min(values))
bars[min_idx].set_edgecolor('black')
bars[min_idx].set_linewidth(2.5)
axes[i].text(bars[min_idx].get_x() + bars[min_idx].get_width()/2,
              bars[min_idx].get_height() + max(values)*0.1,
              '🏆', ha='center', fontsize=14)

plt.tight_layout()
plt.savefig('model_comparison.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Evaluation chart saved!")

```

```

# -----
# CELL 13: FORECAST vs ACTUAL COMPARISON PLOT
# -----

```

```

print("=" * 55)
print("  STEP 8: FORECAST vs ACTUAL PLOT")
print("=" * 55)

```

```

fig, axes = plt.subplots(3, 1, figsize=(15, 13))
fig.suptitle('Forecast vs Actual - All 3 Models', fontsize=15,
fontweight='bold')

```

```

predictions_list = [
    ('Moving Average', ma_predictions, '#e74c3c'),

```

```

        ('SARIMA',          sarima_predictions,  '#f39c12'),
        ('Prophet',        prophet_predictions, '#2ecc71'),
    ]

for i, (name, preds, color) in enumerate(predictions_list):
    axes[i].plot(test_df['date'], actual, label='Actual', color='steelblue',
linewidth=2)
    axes[i].plot(test_df['date'], preds,  label=f'Predicted ({name})',
color=color,
                linewidth=1.8, linestyle='--')
    axes[i].set_title(f'{name} | RMSE:
{results_df[results_df["Model"]==name]["RMSE"].values[0]}',
                    fontweight='bold')
    axes[i].set_ylabel('Incidents')
    axes[i].legend(loc='upper left')
    axes[i].grid(axis='y', alpha=0.3)

    if name == champion:
        axes[i].set_facecolor('#f0fff4')
        axes[i].set_title(f'🏆 {name} [CHAMPION] | RMSE:
{results_df[results_df["Model"]==name]["RMSE"].values[0]}',
                        fontweight='bold', color='darkgreen')

axes[-1].set_xlabel('Date')
plt.tight_layout()
plt.savefig('forecast_vs_actual.png', dpi=150, bbox_inches='tight')
plt.show()
print("✅ Forecast vs Actual plot saved!")

# -----
# CELL 14: KDE HEATMAP (Geospatial Layer)
# -----
print("=" * 55)
print(" STEP 9: KDE GEOSPATIAL HEATMAP")
print("=" * 55)

```

```

import folium
from folium.plugins import HeatMap

# NIR center coordinates (Bacolod City)
nir_center = [10.6713, 122.9511]

# Create base map
incident_map = folium.Map(
    location=nir_center,
    zoom_start=12,
    tiles='CartoDB dark_matter'
)

# Prepare heatmap data [lat, lon, weight]
heat_data = [[row['latitude'], row['longitude'], row['count']]
              for _, row in df.iterrows()]

# Add heatmap layer
HeatMap(
    heat_data,
    radius=15,
    blur=12,
    max_zoom=13,
    gradient={'0.2': 'blue', '0.4': 'lime', '0.6': 'yellow', '1.0': 'red'}
).add_to(incident_map)

# Add title
title_html = '''
<div style="position: fixed; top: 10px; left: 50%; transform:
translateX(-50%);
        background-color: rgba(0,0,0,0.7); color: white; padding: 10px
20px;
        border-radius: 8px; font-size: 16px; font-weight: bold; z-index:
9999;">
    NIR Incident Heatmap – Fire, Crime & Accidents

```

```

</div>
'''

incident_map.get_root().html.add_child(folium.Element(title_html))

# Save heatmap
incident_map.save('nir_incident_heatmap.html')
print("✅ KDE Heatmap saved as 'nir_incident_heatmap.html'!")
print("    Open this file in your browser to see the interactive map.")

# -----
# CELL 15: FINAL SUMMARY REPORT
# -----

print("\n")
print("=" * 55)
print("    FINAL RESULTS SUMMARY")
print("=" * 55)

print(f"""
📌 System          : Geospatial Incident Response System
📌 Scope           : Negros Island Region (NIR), Philippines
📌 Incident Types: Fire, Crime, Accidents
📌 Data Period    : {df_daily['date'].min().date()} →
{df_daily['date'].max().date()}
📌 Train/Test     : 80% / 20% split

📊 ALGORITHM EVALUATION RESULTS:
-----
{results_df.to_string(index=False)}
-----

🏆 CHAMPION MODEL : {champion}
    → This model produced the lowest error scores
    → Recommended for integration into your system

🌐 GEOSPATIAL LAYER : KDE Heatmap

```

- Shows WHERE incidents are concentrated in NIR
- Saved as: nir_incident_heatmap.html

 FILES GENERATED:

- eda_analysis.png
- decomposition.png
- train_test_split.png
- model_comparison.png
- forecast_vs_actual.png
- nir_incident_heatmap.html

```
"""
```

```
print("=" * 55)
print("  ✅ ANALYSIS COMPLETE!")
print("=" * 55)
```